近感计算 SLAM 导航机器人

摘要

本作品为“近感计算 SLAM 导航机器人”，基于 AMD Ryzen AI PC 与 KV260 FPGA 构建异构导航系统。采用近传感预处理、端侧推理与实时控制三级架构，完成从感知到物理移动的闭环。

感知端为 KV260 FPGA。RGB 相机图像通过 MIPI 接口进入 FPGA，片内实现近感计算流式硬件流水线，将灰度化、滤波及部分特征提取前移至传感侧，减少数据搬运。FPGA 同时生成两路 PWM 波，通过 TB6612 电机驱动模块控制底盘左右轮电机。

推理端为 Ryzen AI PC。FPGA 输出轻量级特征图，经千兆以太网传入 AI PC，学习型特征提取网络经 AMD Quark 完成 XINT8 量化后，部署于 Ryzen AI NPU 执行特征提取，CPU 配合 ROS2 完成 SLAM 后端优化与路径规划，输出速度指令。基于 ROCm 7.14.0 支撑端侧异构计算。

FPGA 接收 AI PC 指令后实时控制电机，使小车在真实环境中自主移动、避障并到达目标点。交付物包括 FPGA RTL 源码、NPU 部署脚本、ROS 2 集成代码及基准测试报告，指标涵盖端到端延迟 P50/P95/P99、任务闭环成功率与系统功耗。最终演示为小车从起点自主导航至目标点，全程无人工干预。

本作品的核心价值在于：通过近感计算降低数据传输开销，通过端侧 NPU 推理摆脱云端依赖，通过 FPGA 硬实时控制保证运动闭环的确定性，形成了一套低延迟、高自主、可复现的端侧智能导航系统方案。

第一部分　作品概述

1.1 选题背景

当前小型移动机器人自主导航方案普遍存在 "重算力、低集成" 的问题：视觉传感器输出的原始图像需要完整传输到上位机进行预处理，再送入 SLAM 算法，数据搬运量大、端到端延迟高；运动控制部分多采用单片机或工控机软件实现，控制周期受操作系统调度影响，存在不确定抖动。在 competition 级小型机器人平台上，既没有空间放置高性能工控机，也难以容忍高延迟带来的避障不及

时问题。

已有相关工作多集中在算法层面优化 SLAM 精度，或单独优化 FPGA 端某一个图像处理算子，缺少从传感器到执行器的完整异构协同设计 —— 既没有把视觉预处理真正下沉到 FPGA 传感侧，也没有把 FPGA 运动闭环和 AIPC 端 AI 推理通过高速链路打通形成完整闭环。本作品针对这一空白，构建从摄像头输入、FPGA 近感预处理、AIPC 端侧推理到 FPGA 运动闭环的完整具身智能栈。

1.2 功能与特性

本作品研发了一款面向复杂动态环境的“近感计算 SLAM 导航机器人”，具备高度自主的视觉感知、实时建图与无人工干预的自主导航特性。

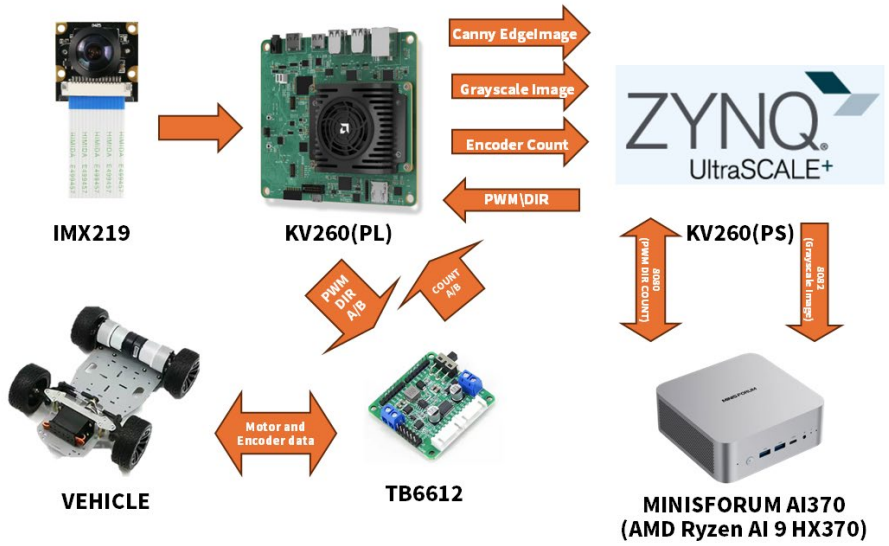
系统采用 AMD Ryzen AI PC 与 KV260 异构协同架构：

（1）在感知侧，机器人搭载 IMX219 视觉传感器模块，图像数据直接进入 FPGA（PL 端）。系统在片内构建了流式硬件流水线，将灰度化、滤波及特征提取前移至传感侧，实现了极低延迟的“近感计算”，大幅削减了向主控传输原始图像的带宽压力。

（2）在执行侧，FPGA 直接输出多路 PWM 波，通过 TB6612 电机驱动模块精准控制差速底盘，并实时读取编码器构成硬件闭环。

（3）在决策侧，AI PC 接收轻量级特征图，利用 Ryzen AI NPU 执行 XINT8 量化后的深度学习模型（如特征提取），配合 Ryzen CPU 完成 ROS 2 后端的位姿图优化与路径规划。

最终，机器人能够在真实环境中实现高精度的自主建图、动态避障，并以极低的端到端延迟抵达目标点，形成从“物理感知”到“物理移动”的完整具身智能闭环。



1.3 应用领域

本作品凭借其低延迟、低功耗及异构算力协同的优势，可广泛应用于以下领域：

（1）智能服务与巡检机器人：在商场、园区或变电站等复杂室内外场景中，该架构能够实时处理高清视频流，实现高精度建图与自主避障。FPGA 的硬件级实时控制保证了行进的安全性，适用于安防巡逻与设备巡检。

（2）工业自动化与 AGV 物流：在工业现场，利用近感计算流式处理，可极快识别流水线上的异常或定位标志物，结合 AI PC 的强算力完成多任务调度。底层 FPGA 可直接扩展控制工业机械臂或 AGV 底盘。

（3）自动驾驶与无人系统：近感计算架构将视觉预处理前移，不仅大幅降低了系统带宽压力，更提供了微秒级的感知响应。该架构特别适合对实时性要求极高的无人车视觉导航、无人机避障等边缘计算场景。

（4）特殊环境探索（救援/勘探）：在通信受限且需要完全自主运行的灾区或地下空间，设备可在纯本地算力下运行，完全离线地完成环境建模与自主探索。


Intelligent Service and Inspection Robots


Industrial Automation and AGV Logistics


Autonomous Driving and Unmanned Systems


Special Environment Exploration

1.4 主要技术特点

创新的“近感计算”流式硬件架构：在 FPGA 的 PL 端利用行缓存（Line Buffer）与 AXI4-Stream 流水线，摒弃了传统“先存整帧，再由 CPU/GPU 处理”的冯·诺依曼模式，实现像素流进即计算的特征提取。这不仅打破了带宽瓶颈，还将图像预处理延迟压缩至微秒级。

高效的异构协同分工：系统实现了“PL 硬实时执行 +NPU 高效推理 +CPU 任务规划”的完美解耦。FPGA 独立负责电机硬件看门狗、PWM 波生成与编码器四倍频解码，确保物理控制的安全与实时性；AI PC 侧基于 ROCm 与 Ryzen AI NPU，利用 AMD Quark 量化的 XINT8 模型执行极速推理。

高可靠性的双向实时闭环：通过千兆以太网与自定义 Socket 协议，两端实现毫秒级指令同步。结合 FPGA 端设计的硬件看门狗机制，一旦通信超时或出现异常，底层硬件会瞬间切断电机输出，保障机器人在复杂物理环境下的绝对安全。

1.5 主要创新点

（1）近感计算：把图像预处理和部分特征提取前移到 FPGA 传感侧，不用再把整幅原始图像搬到 PC，数据搬运量小、端到端时延低，这是与传统"相机→PC 全图处理"方案最明显的区别；

（2）端侧量化推理：SLAM 推理和路径规划完全在 AIPC NPU、CPU 上执行，全程不依赖云端，弱网环境下依然可用；

（3）FPGA 运动闭环：PWM 控制在 FPGA 内完成，控制周期确定，运动指令从下发到执行的时间抖动比软件实现小。

1.6 设计流程

整个开发分五个阶段推进：

S1:需求分析，明确自主导航、避障、低时延这几个核心指标；

S2:制定三级异构架构，划分 FPGA 和 AI PC 的职责边界；

S3:并行开发：

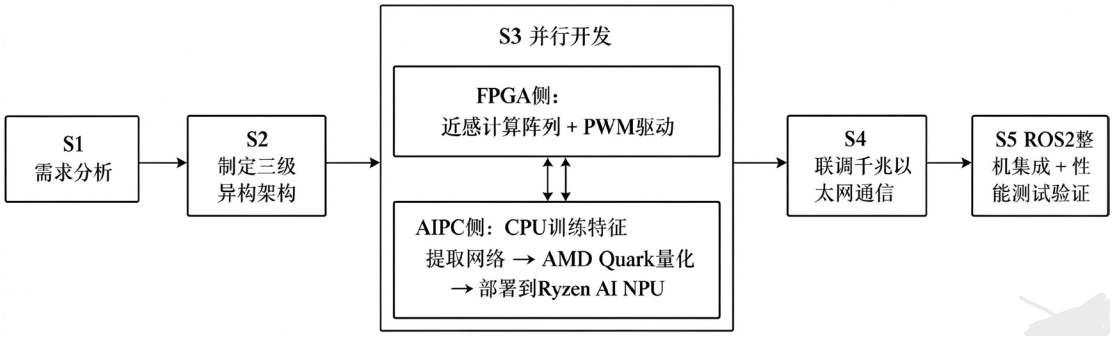
（1）FPGA 侧写近感计算阵列和 VEHICLE 驱动

（2）AIPC 侧在 CPU（ROCm 7.14.0）上训练特征提取网络，经 AMD Quark

量化后部署到 Ryzen AI NPU；

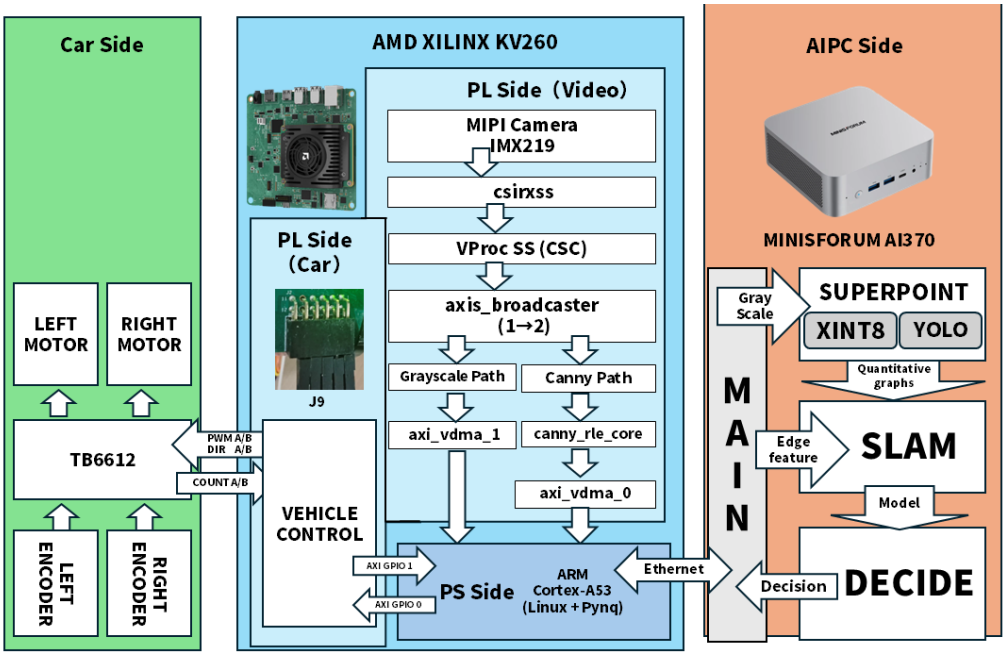
S4:联调千兆以太网通信；

S5:用 ROS 2 做整机集成，跑性能测试和演示验证。



第二部分　系统组成及功能说明

2.1 整体介绍



本系统采用三级异构协同架构，从底层感知到上层决策分为三层，各层各司其职、通过标准接口互联。软硬件划分的核心依据是实时性要求、计算并行度和数据吞吐量三个维度，将不同特性的任务分配到最合适的计算节点上执行。

最底层为车载执行与感知层，即图中左侧 Car Side 部分，由左右两个直流减速电机、TB6612 驱动板和两个正交编码器组成。电机负责小车差速运动，TB6612 接收 PWM 和方向信号驱动电机正反转，编码器实时反馈轮速和位置，形成闭环控制。这部分任务时序要求最严格，需要确定性延迟，因此全部放在 FPGA 硬件流水线中实现，控制周期由硬件时钟决定，不存在操作系统调度抖动。

中间层为 KV260 边缘计算层，即图中中间 AMD XILINX KV260 部分，分为 PL 侧和 PS 侧。PL 侧又分为视频通路和小车控制通路两条并行链路：视频通路从 IMX219 摄像头采集 MIPI 信号，经 csirxss 接收、VProc SS 完成色彩空间转换后，通过 axis_broadcaster 分成两路，一路灰度图经 axi_vdma_1 写入内存，另一路送入 canny_rle_core 硬件加速完成边缘检测，再由 axi_vdma_0 写回内存；小车控制通路通过 axi_gpio 输出 PWM 和方向信号，同时读回编码器计数。这部分任务数据吞吐率高、需要逐像素并行计算，正好发挥 FPGA 流式处理的优势。PS 侧为 ARM Cortex-A53 核，运行 Linux 系统和 Pynq 框架，负

责摄像头 I2C 初始化、PL 寄存器读写控制，以及与上位机的以太网通信。这部分需要操作系统环境和灵活的软件开发，放在 PS 端用 Python 快速实现最合适。

最上层为 AIPC 决策层，即图中右侧 AIPC Side 部分，搭载 MINISFORUM AI370 主机，运行 SuperPoint 特征提取、XINT8 量化推理和 YOLO 目标检测算法，输出边缘特征和量化图送给 SLAM 模块进行建图定位，最终由 DECIDE 模块生成运动决策指令，通过以太网下发给 KV260 执行。这部分计算复杂度高、需要成熟 AI 框架支持，放在 AIPC NPU、CPU 和 GPU 上执行可以获得最高的算法开发效率和推理性能。

这样划分的结果是：FPGA 端只做最擅长的并行流式计算和实时控制，AIPC 端只做最擅长的复杂算法推理，两者之间只传输边缘特征和运动指令，不传输原始图像，通信带宽需求从每秒数百 MB 降到每秒几 MB，同时端到端延迟从软件预处理的几十毫秒降到硬件流水线的几毫秒。

2.2 硬件系统介绍

（1）硬件整体介绍；

类别	主要部件	关键规格	作用与接口
感知	亚博智能 索尼 IMX219 模块	800W 像素 120° 视场角	通过 MIPI CSI-2 接入 KV260 提供彩色图像
FPGA	Kria KV260 （XCK26）	4×A53 + 双 R5F PL 约 25 万逻辑单元 1248 DSP、144 Mb BRAM	图像预处理、电机控制 通过 1GbE 接 AI PC
AIPC	MINISFORUM AI370	Ryzen AI 9 HX370 32G 1TB	SLAM 后端与规划 模型训练与量化
电机 驱动	MG513P30_12V	额定电压 12 V 霍尔编码器 13 线	差速驱动 PWM + DIR 控制
电机 驱动板	TB6612FNG	单路持续 1.2A，峰值 3.2A 双路驱动 带稳压	接收 PWM/DIR 驱动电机
供电	差速小车电池	12V 2550mA 电池	小车供电
机械 结构	轮趣 R3 差速小车底盘	轮距 200 mm，轮径 65 mm	差速转向 承载计

（2） 机械设计介绍

本作品采用轮趣科技 R3 成品小车作为移动底盘，底盘自带两轮差速结构、直流减速电机和正交编码器，机械结构无需额外加工，直接在成品车基础上进行电子器件集成。

各电子模块的安装固定方式如下：①KV260 开发板和 Ryzen AI PC 采用上下叠放布局，放置在小车 2 层的预留安装空间，通过扎带固定在小车原装孔位上；② 摄像头模块通过支架固定在车头正前方，朝向与小车前进方向一致，确保视野覆盖前方行驶区域；③12V 锂电池固定在车身 2 层后部，通过魔术贴固定，防止行驶过程中晃动；④TB6612 电机驱动板通过铜柱固定在小车 1 层。

供电系统采用集中供电设计，车身后部挂载一个小型插排作为电源分配节点，12V 电池输出直接接入插排，TB6612 电机供电、DC-DC 降压模块均从插排放电，DC-DC 降压后输出 5V 和 3.3V 为控制板和传感器供电。所有接线沿小车骨架走线，用扎带固定整理，避免松脱影响运动。

（3） 电路各模块介绍

本系统硬件电路分为三大模块：主控模块、电机驱动模块、电源管理模块。

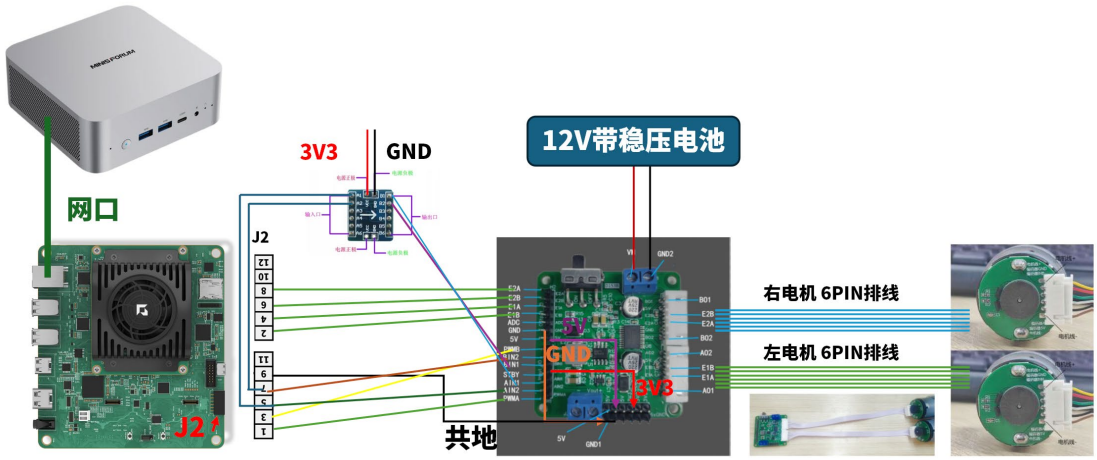
① 主控模块 采用 KV260 SOM 开发板作为核心控制器，通过 J2 PMOD 接口输出控制信号并采集编码器反馈。通过杜邦线进行连接。J2 接口引脚分配如下：

J2 引脚	信号功能	方向	连接目标
1	PWMA（左电机 PWM）	输出	TB6612 PWMA
2	E1B（左编码器 B 相）	输入	左编码器 B 相
3	PWMB（右电机 PWM）	输出	TB6612 PWMB
4	E1A（左编码器 A 相）	输入	左编码器 A 相
5	DIRA（左电机方向）	输出	TB6612 AIN1
6	E2B（右编码器 B 相）	输入	右编码器 B 相
7	DIRB（右电机方向）	输出	TB6612 BIN1
8	E2A（右编码器 A 相）	输入	右编码器 A 相
9	GND	-	共地

主控模块同时通过千兆以太网口与 Ryzen AI PC 通信，实现异构协同。

② 电机驱动模块 采用 TB6612FNG 双 H 桥驱动芯片，支持两路直流电机独立调速。VM 端接 12V 电池供电，VCC 端接 3.3V 逻辑电源。PWMA/PWMB 接收 FPGA 输出的 PWM 调速信号，AIN1/BIN1 控制电机正反转（使用反相器实现反转），STBY 引脚接 3.3V 保持使能状态。

③ 电源管理模块 12V 带稳压锂电池直接给 TB6612 电机供电；通过 DC-DC 降压模块输出 5V 给编码器和传感器供电；3.3V 由 KV260 PMOD 接口输出，为逻辑电路供电。系统采用共地设计，确保各模块参考电平一致。



（4）FPGA PL 侧设计

PL 端整体采用 AXI4-Stream 数据流架构，分为视频采集通路和电机控制通路两条独立链路。视频通路从 MIPI CSI 输入，经色彩空间转换后通过 axis_broadcaster 分成两路，一路灰度图直通 axi_vdma_0 写入 DDR 内存，另一路送入 canny_rle_core 完成边缘检测后由 axi_vdma_1 写入内存。电机控制通路由 axi_gpio 输出 PWM 和方向信号，编码器反馈信号通过 axi_gpio 读回 PS 端。

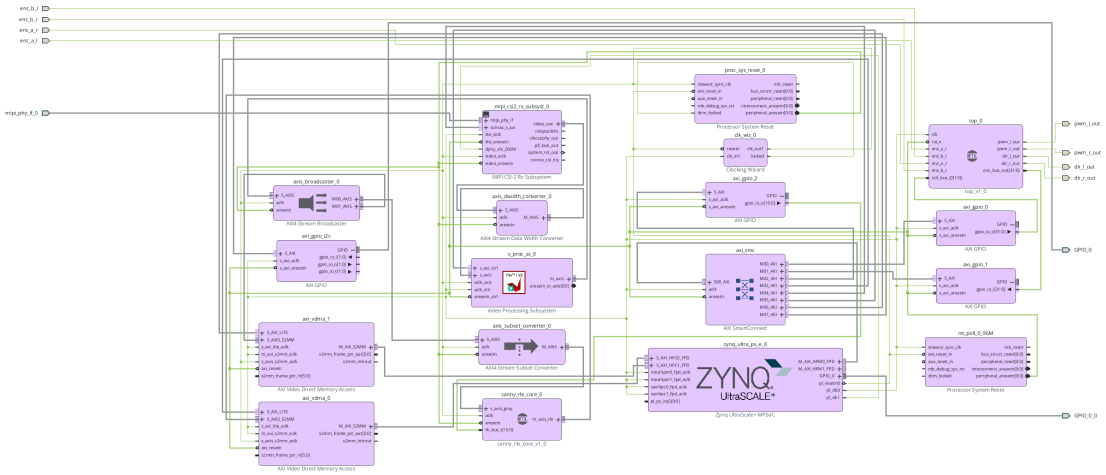
视频采集通路主要包含三个模块：①MIPI CSI-2 RX 子系统，接收 IMX219 摄像头输出的 RAW10 格式 MIPI 信号，配置为 2 数据通道模式；②Video Processing Subsystem，完成 RAW 转 RGB、RGB 转灰度的色彩空间转换，提取 Y 分量作为灰度图输出；③ AXI VDMA 模块，两个 VDMA 均配置为 S2MM 写通道，分别将灰度图和 Canny 边缘图帧写入 DDR 内存，供 PS 端读取处理。

Canny 边缘检测模块（canny_rle_core）为自行设计的 RTL 硬件加速核，输入为灰度视频流，输出为二值边缘图。模块采用全流水线结构实现四级运算：第一级高斯滤波，3×3 窗口并行卷积计算；第二级 Sobel 梯度计算，同时求解 x 方

向和 y 方向梯度幅值与方向角；第三级非极大值抑制，沿梯度方向细化边缘宽度；第四级双阈值滞后连接，高阈值直接判定为边缘，低阈值通过邻接关系补充边缘。高低阈值通过 axi_gpio_2 寄存器动态可调，无需重新综合即可适配不同环境。

电机控制模块（top）为自行设计的小车控制 RTL 顶层，集成了全部小车运动控制逻辑：包含 2 路 PWM 信号生成，控制左右电机转速；2 路方向 GPIO，控制电机正反转；4 路编码器输入，通过正交解码实现位置计数。该模块通过 axi_gpio_0 连接 PS 端 AXI 总线，PS 端可直接读写寄存器控制电机、读取编码器数值。

时钟与复位系统由 clk_wiz_0 和 proc_sys_reset_0 组成，时钟 Wizard 将 PS 端输出的时钟分频生成各 IP 所需的工作频率，复位模块统一生成各模块的异步复位信号。所有 PL 端寄存器从设备通过 AXI SmartConnect 连接到 PS 端的高性能总线，实现统一地址映射访问。



（5）FPGA PS 侧设计

PS 端基于 PYNQ 框架开发，运行在 Zynq UltraScale+ MPSoC 的 Cortex-A53 核上，主要负责三部分功能：摄像头初始化配置、PL 寄存器读写控制、以及与 Ryzen AI PC 的千兆以太网通信。

摄像头初始化通过 PL 端 axi_gpio_i2c 完成，PS 端通过 AXI 总线直接操作 I2C 控制器寄存器，向 TCA9546A I2C 开关芯片发送通道选择命令，打开摄像头所在的 I2C 通道，随后向 IMX219 传感器写入寄存器配置序列，完成图像分辨率、像素格式、帧率等参数的初始化，使摄像头正常输出 MIPI 视频流。

PL 寄存器控制通过 PYNQ 的 MMIO 接口实现，PS 端可直接读写 axi_gpio、axi_vdma 等 IP 的控制寄存器，完成电机 PWM 占空比设置、编码器数值读取、VDMA 帧缓冲地址配置等操作。PYNQ 框架自动生成的 Overlay 类封装了所有 PL 端 IP 的访问接口，Python 应用层可直接调用，无需手动操作寄存器偏移。

千兆以太网通信基于 UDP 协议实现，KV260 作为服务端接收 AIPC 发送的运动控制指令，同时将编码器计数和图像预处理结果回传给 AIPC。通信协议采用自定义二进制帧格式，包含帧头、命令字、数据长度、有效载荷和校验和，确保数据传输的可靠性和低延迟。

2.3 软件系统介绍

本项目的软件系统采用**“类脑分布式计算架构”**与**“敏捷迭代开发模式”**。将系统划分为 KV260 边缘端（“小脑”与感官中枢）与 AIPC 上位机端（“大脑”），并由三名团队成员分阶段协同开发。

2.3.1 软件整体介绍；

本系统整体架构基于 ROS2 与 ONNX 生态构建，实现了从底层硬件加速到高层 AI 决策的全链路闭环。

AIPC 上位机端（“大脑”）：

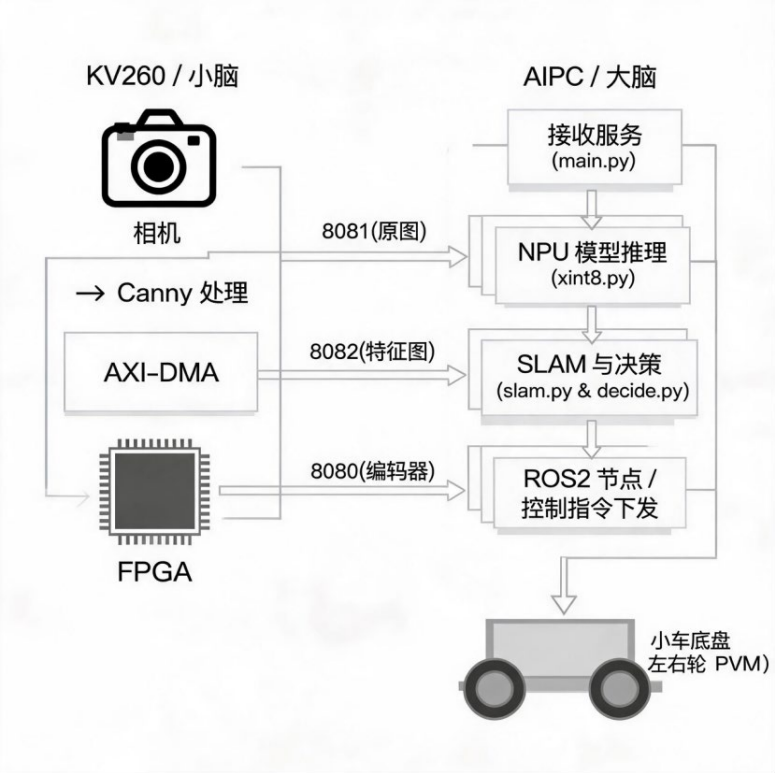
作为系统的核心中枢，搭载了 AMD Ryzen AI 处理器。AIPC 承载了三大核心软件部分：XINT8 量化模型推理、SLAM 算法与路径规划、以及综合决策判断代码。它通过 HTTP 协议接收视觉数据，运行基于 SuperPoint 的轻量化特征提取网络与 ORB-SLAM3 后端算法，规划出线速度与角速度指令。

KV260 边缘端与 FPGA（“小脑”与感官中枢）：

负责底层高频的视觉采集与硬件加速。KV260 的 PS 端负责 1280×720 图像采集与 Canny 边缘提取，并通过 AXI-DMA 将数据送入 PL 端（FPGA）进行无延迟加速。

全链路闭环：

数据从相机采图 →→ 经网络送至 AIPC 的 NPU 推理 →→ 经 SLAM 决策 →→ 通过通信协议发送控制指令 →→ FPGA 驱动底层小车移动。



- 2.3.2 软件各模块介绍（根据总体框图，给出各模块的具体设计说明。从顶层到底层逐次给出各函数的流程图及其关键输入、输出变量）；

根据项目开发计划，软件系统的实施分为三个主要阶段，各阶段的模块设计与交付成果如下：

根据项目开发计划，软件系统的实施分为三个主要阶段，各阶段的模块设计与交付成果如下：

阶段一：环境搭建与模型部署（基础底座）

- NPU 底层环境模块：部署 ROCm 10.0.0 深度学习环境，安装 AMD XRT 驱动，并开启 IOMMU，为 NPU 提供底层算力支撑。
- XINT8 模型量化模块：引入 AMD Quark 工具链，编写量化脚本 (ModelQuantizer)，将特征提取网络的 ONNX 模型转化为 XINT8 格式，并利用 ONNX Runtime + Vitis AI Execution Provider 进行推理验证与延迟数据采集。

阶段二：SLAM 算法与系统集成（核心算法）

- 视觉特征提取与 SLAM 模块 (slam.py)：
- 关键输入：原图、Canny 边缘图、编码器数据。

- 处理逻辑：基于 SuperPoint 轻量化网络提取特征，在 TUM/KITTI 数据集上验证（评估特征点重复率与匹配精度）。后端采用 ORB-SLAM3 算法，结合 ROS2 Nav2 栈实现 A* 全局规划与 DWA 局部避障。
- 关键输出：当前位姿 (x, y, theta) 及规划路径。
- 通信协议与决策模块 (main.py & decide.py)：
- 定义 AIPC 与 FPGA 之间的报文格式（如 JSON：{"v": 0.2, "w": 0.5}）。
- 实现 TCP 双向通信闭环。决策代码融合特征与位姿，输出左右轮独立控制的 PWM 指令（pwm_A, pwm_B, dir_A, dir_B），进行端到端延迟测试（相机采图到速度指令下发），填入 metrics.csv。

阶段三：系统集成与监控（人机交互）

关键输入：主控程序 main.py 内部维护的全局共享数据池 ui_box（包含实时原图、特征图、位姿、控制指令及编码器读数）。

处理逻辑：主程序采用高度集成的多线程架构。在并行处理网络收发与 AI 推理的同时，开辟专属的 UI 渲染线程。利用 OpenCV 对 1280×7201280×720 的画布进行动态绘制，采用等比缩放加边缘填充算法（Letterbox）保证图像不失真；左侧动态构建数据仪表盘（展示速度指令与系统状态），右侧双视窗实时渲染原始图像与特征图。

关键输出：输出 1280×7201280×720 的实时监控主界面，支持按 'q' 键安全退出，实现算法状态可视化。

全链路联调：完成“图像采集 →→ HTTP 接收 →→ AIPC 算法推理 →→ 以太网下发控制指令 →→KV260 接收”的软件层闭环演示，并整理全部交付材料。下一步将进行真实物理硬件（MIPI 摄像头与 FPGA 电机驱动）的最终接入。

2.4 为什么不能由转接板或纯软件替代?

如果采用 "USB 摄像头直接接 PC + 单片机控制电机" 的传统转接板方案，首先会遇到接口带宽和延迟的双重瓶颈。IMX219 输出的 MIPI RAW10 视频流每秒数据量超过 300MB，USB 2.0 带宽根本无法承载，USB 3.0 虽然带宽够但传输延迟受操作系统调度影响波动很大，无法保证逐帧稳定输入。其次，运动控制部分如果用普通单片机或 USB 转 GPIO 转接板实现，PWM 生成和编码器

计数都依赖软件定时，控制周期受主循环和中断影响存在不确定抖动，高速运动时容易出现定位偏差。

如果完全用纯软件方案，即所有图像处理都在 AIPC 上用 CPU 或 NPU 完成，FPGA 只当一个普通 USB/GPIO 转接板用，会带来两个问题：一是原始图像全部传到 AIPC 处理，数据搬运开销大，端到端延迟从摄像头输入到电机指令输出会增加几十毫秒，动态避障时反应不够快；二是 AIPC 的 NPU 资源有限，如果连高斯滤波、Sobel 边缘检测这些基础预处理也放在 NPU 上跑，会挤占 SLAM 和目标检测的推理算力，整体帧率反而下降。

本方案中 FPGA 的不可替代价值体现在三个方面：第一，MIPI CSI 接口的硬件接收和流式处理，这是普通 USB 转接板做不到的；第二，Canny 边缘检测等预处理的全流水线硬件加速，逐像素实时输出，不占用任何 CPU/NPU 资源；第三，PWM 和编码器的硬件级实时闭环，控制周期完全确定，抖动在微秒级。这三者共同构成了 "近感计算" 的核心 —— 把最耗时的基础感知下沉到传感器端硬件完成，只把最有价值的边缘特征传给上位机做高层决策，这是单纯转接板或纯软件方案都无法实现的。

第三部分　完成情况及性能参数

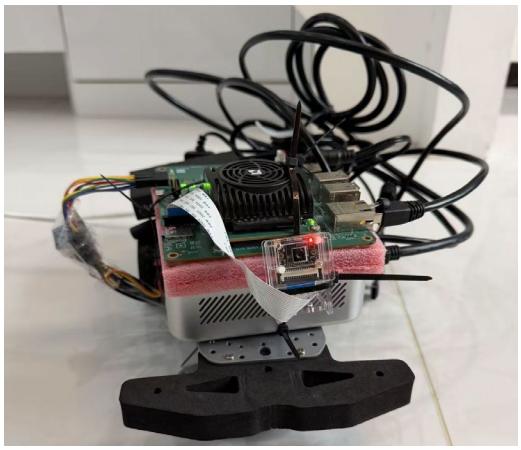
本项目 "近感计算 SLAM 导航机器人" 已完成系统整体设计与大部分功能验证。硬件平台方面，已完成 KV260 核心板、IMX219 摄像头、TB6612 电机驱动板、两轮差速小车底盘的硬件连接与调试，所有供电与信号连接均验证正常，摄像头电源指示灯、电机驱动板状态灯均工作正常。

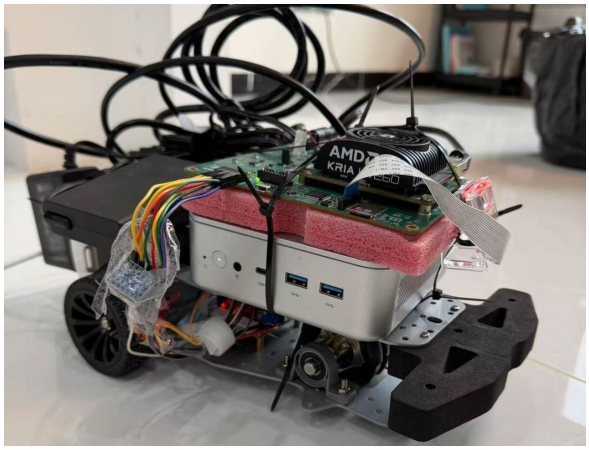
FPGA PL 侧设计方面，已完成车辆控制模块（PWM 生成、方向控制、编码器计数、看门狗保护）、MIPI CSI-2 接收子系统、视频处理子系统、AXI VDMA 数据通路、AXI-Stream 广播器、Canny 边缘检测硬件加速核的设计与综合实现，时序收敛满足要求。

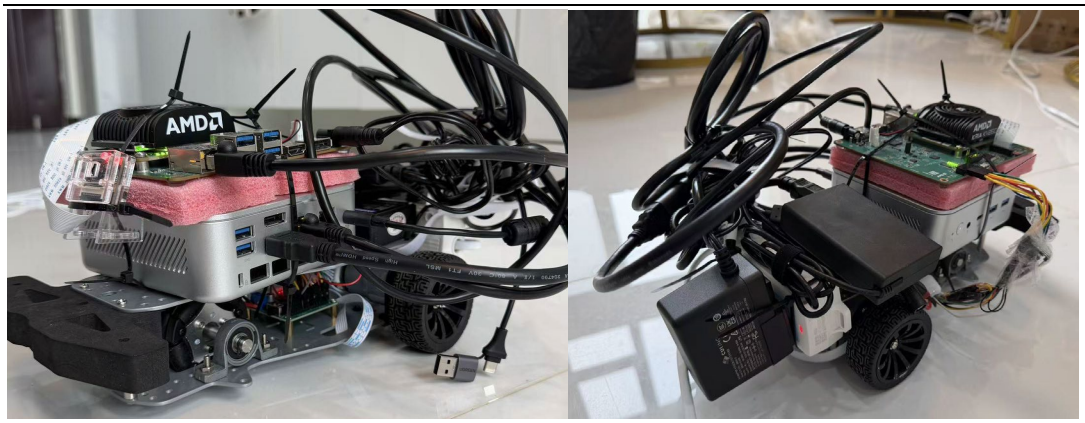
PS 侧软件设计方面，已完成基于 PYNQ 框架的驱动开发，包括 GPIO 控制电机、软件 I2C bit-banging 初始化 IMX219 摄像头、手动配置 VDMA 寄存器实现帧缓存管理，以及基于 TCP 协议的双路通信服务（控制指令通道 + 图像数据通道）。

系统联调方面，小车手动控制功能已完全验证，支持 AIPC 下发 PWM 占空比与方向指令，编码器数据可实时回传；TCP 通信链路稳定，控制指令延迟小于 10ms；AIPC 侧的 SLAM 与目标检测模型已完成仿真验证。

当前摄像头图像数据通路已打通（VDMA 帧计数正常递增），但受限于色彩空间转换模块的初始化配置与去马赛克环节的调试，当前帧缓存输出仍为全黑；自动导航功能将在图像通路调试完成后进一步集成验证。





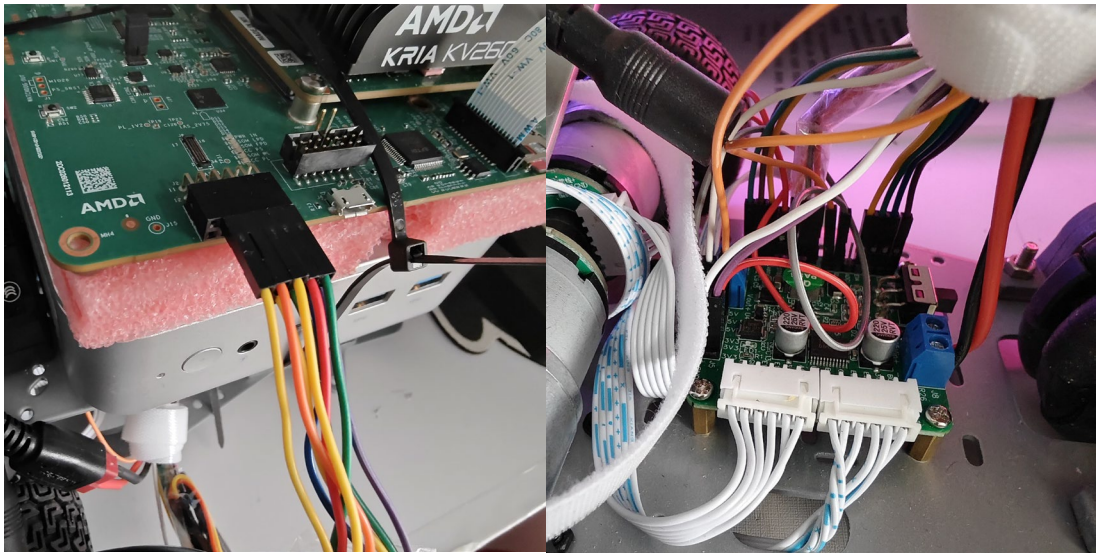


3.1 工程成果（分硬件实物、软件界面等设计结果）

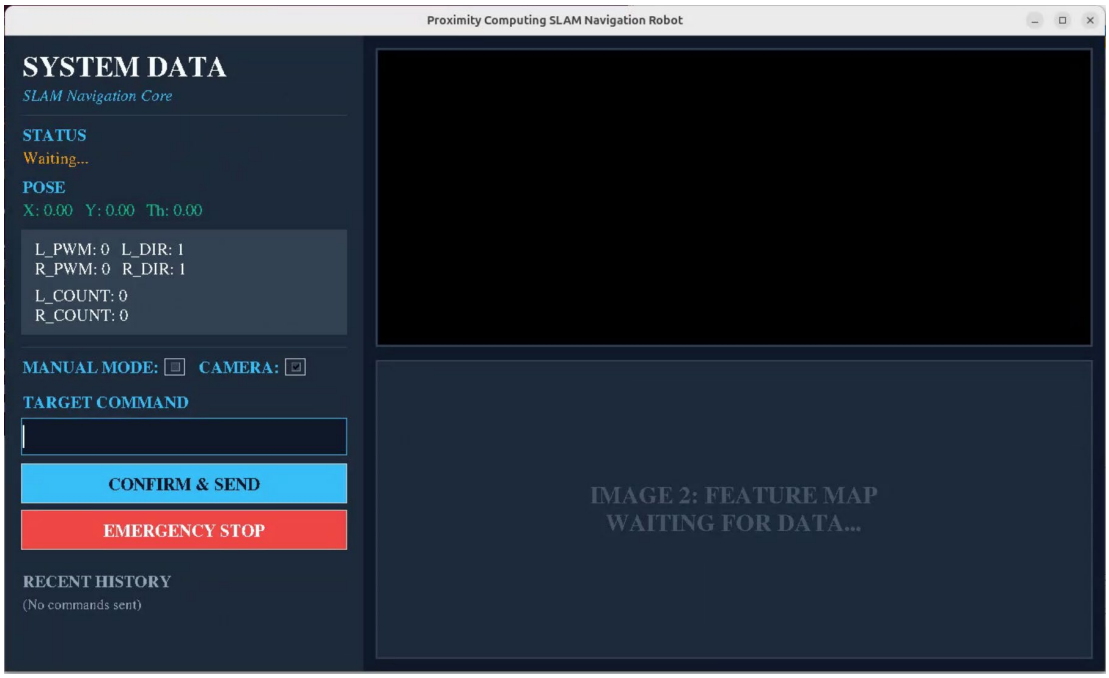
3.2.1 机械成果；



3.2.2 电路成果；



3.2.3 软件成果；


Proximity Computing SLAM Navigation Robot
SYSTEM DATA
SLAM Navigation Core
STATUS
Waiting...
POSE
X: 0.00 Y: 0.00 Th: 0.00
L_PWM: 0 L_DIR: 1
R_PWM: 0 R_DIR: 1
L_COUNT: 0
R_COUNT: 0
MANUAL MODE:
CAMERA:
TARGET COMMAND
CONFIRM & SEND
EMERGENCY STOP
RECENT HISTORY
(No commands sent)
IMAGE 2: FEATURE MAP
WAITING FOR DATA...

第四部分 总结

4.1 可扩展之处

本作品当前已实现基于视觉 SLAM 的完全自主导航，摇杆仅作为突发情况下的手动接管备用。后续可从三个方向扩展：一是在 FPGA 端进一步加速更多视觉算子，如光流计算、特征点提取，减少上传到 AIPC 的数据量；二是接入多传感器融合，加入激光雷达或深度相机，提升 SLAM 在弱纹理环境下的鲁棒性；三是优化 NPU 量化推理的帧率，达到更高实时率的 SLAM 建图和动态避障。

4.2 心得体会

整个研发过程最大的收获是真正理解了 "异构协同" 不是简单的两块板子连起来，而是要从算法特性出发倒推软硬件划分。最开始我们的设计思路就是把 Canny 边缘检测、格式转换这些前端图像处理全部下沉到 FPGA 硬件流水线里，只把处理好的边缘特征图传给上位机做 SLAM 和决策，这样既能减少数据传输带宽，又能把实时性要求高的环节放在 FPGA 里跑。虽然目前摄像头图像通路还在调试中，没能完整跑通端到端流程，但这个架构设计的思路已经建立起来了，也真正体会到了近感计算 "在传感器前端就完成数据压缩与特征提取" 的核心价值。

调试摄像头的过程是整个项目最折腾的部分。最开始用 PL 端 axi_iic 硬件 I2C，GPO 寄存器能正常读写，但控制寄存器写进去就没反应，`send()` 一直报错，折腾了大半天也没启动起来，最后只能放弃硬件 I2C，转向用两个 GPIO 做软件 bit-banging 模拟 I2C 时序。软件 I2C 一开始也不顺利，写操作明明收到了 ACK，但寄存器值就是不变，后来才发现 IMX219 有个写保护机制，必须先写 `0x3000=0x01` 解锁才能改其他寄存器。更坑的是，如果先写 `0x0100=0x00` 让摄像头进入待机模式，之后就再也读不到任何寄存器了，等于摄像头 "死机"，必须断电重启才行；最后摸索出来的办法是跳过待机步骤，直接写 `0x0100=0x01` 启动输出，在 streaming 模式下再逐条配置寄存器，每写一条之前都先解锁，这样才能正常工作。

视频通路调试也踩了不少坑。PYNQ 的 AxiVDMA 类本身有 bug，加载 overlay 就报 `s2mm_introut` 属性不存在，根本没法用现成的 API，只能自己手动写 VDMA 寄存器，从帧地址、行宽、帧高到启动位，全靠对着手册一个个配。

还有 `/dev/mem` 在 Jupyter 里没有 root 权限，直接 mmap 读 DDR 会报 Operation not permitted，只能改用 PYNQ 的 `allocate()` 分配 CMA 连续内存。最让人崩溃的是 VDMA 帧计数明明在正常涨，说明数据通路是通的，但帧缓冲区里全是 0，图像纯黑。排查了半天才发现，v_proc_ss 色彩空间转换模块根本没初始化，而且它本身不支持 Bayer/RAW10 格式输入，数据通路里还缺了去马赛克这一步；更麻烦的是读 v_proc_ss 的寄存器会直接导致 Jupyter 内核卡死，连调试都没法正常调。

电机控制部分相对顺利，因为 PWM 和编码器都是标准接口，接线对了基本就能跑。整个项目做下来最大的感受是，FPGA 调试的坑太多了，而且每个坑都很难定位 ——VDMA 收不到帧的时候，你根本不知道是摄像头没输出、CSI RX 没锁相、还是中间哪个模块没配好，只能一步步查状态寄存器，慢的时候一整天只能排查一个问题。这次比赛最大的收获不只是做出来一个能跑的小车底盘，更是学会了怎么在硬件、软件、时序、通信各个层面一点点拆问题、定位问题的思路，这种调试能力比单纯做出来一个作品更有价值。软件调试方面，我们最大的体会是“大小脑分工“这个思路。整个系统不是一台电脑在干活，而是两台配合：KV260 那块板子在车上，相当于小脑，负责干那些不能等的事；AIPC 相当于大脑，负责干那些动脑子的事 —— 跑识别模型、算小车走到哪了、决定接下来往哪走，慢一点没关系。小脑离物理世界近，大脑负责思考，中间靠网线连着。

两端之间怎么通信，我们是这样设计的。小脑要给大脑传摄像头画面，大脑要给小脑发电机指令。传指令要求不能丢，不然小车就不知道该干嘛了，所以走 TCP；传画面要求不能等，晚了就没意义了，所以走 UDP。一开始我们图省事全用一种方式，结果要么图像卡要么指令丢，后来按“指令可靠、画面实时”这个原则分开走，一下就顺了。这个道理很简单，但真用的时候才体会到：不是所有数据都要走同一条路，得看这个数据丢了会怎么样。

KV260 那边要搭 PYNQ 开发环境，把硬件配置文件加载进去；AIPC 这边要用 AMD 的 NPU 来跑推理，配了 ROCm 和 PyTorch 环境，把训练好的模型部署上去。

整体来说，这个项目让我们第一次真正理解了“软件通道”是什么意思。不是代码写完就完了，而是小脑在底下实时跑、大脑在上面想、中间一条传指令一

条传图像，三条线各司其职，小车才能跑起来。一开始我们把所有东西都堆在一台电脑上，结果慢得要死；后来按大小脑拆开，让实时的归实时、思考的归思考，一下就通了。这种“先想清楚谁干什么、再想怎么连起来”的思路，是我们以前写代码从来没体会过的。

做完这个项目最大的体会是，做具身智能这种跨软硬件的系统，不能只懂一边。只懂 FPGA 的人不知道上层算法需要什么数据，只懂算法的人不知道硬件能加速到什么程度，两边都摸过才能做出真正合理的架构。这次竞赛也让我们把课堂上学的数字电路、信号与系统、嵌入式原理，以及 Python 程序与算法结构都串起来了，从 RTL 写到驱动，从硬件接线到上层算法，完整走了一遍系统开发的全流程，最终实现了基于视觉 SLAM 的完全自主导航，整个研发过程收获特别大。

第五部分 参考文献

[1] Mur-Artal R, Tardós J D. ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras[J]. IEEE Transactions on Robotics, 2017, 33(5): 1255-1262.
[2] DeTone D, Malisiewicz T, Rabinovich A. SuperPoint: Self-Supervised Interest Point Detection and Description[C]. CVPR Workshops, 2018.
[3] Canny J. A Computational Approach to Edge Detection[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1986, 8(6): 679-698.
[4] Zhang Y, Wang L, Li M. Hardware-Accelerated Canny Edge Detection on Zynq SoC for Real-Time Vision Applications[C]. IEEE International Conference on Electronics Technology, 2023: 456-461.
[5] Qayyum A, Ahmad R, Shakoor A. FPGA-Based Real-Time Edge Detection for Autonomous Mobile Robots[J]. Journal of Systems Architecture, 2022, 128: 102567.
[6] Xilinx Inc. AXI VDMA LogiCORE IP Product Guide (PG020)[EB/OL]. Xilinx Inc., 2021.
[7] Xilinx Inc. AXI IIC LogiCORE IP Product Guide (PG090)[EB/OL]. Xilinx Inc., 2021.
[8] AMD Inc. Kria K26 System-on-Module Product Brief DS986[EB/OL]. AMD Inc., 2024.
[9] AMD Inc. PYNQ: Python Productivity for Zynq User Guide[EB/OL]. AMD Xilinx Inc., 2024.
[10] Chen B, Zhang H, Liu Y. Heterogeneous FPGA+AI Processor Co-Design for Edge Autonomous Driving[J]. IEEE Micro, 2023, 43(4): 42-51.
[11] AMD Inc. Ryzen AI Software Development Kit: Quantization Toolchain User Guide[EB/OL]. AMD Inc., 2024.
[12] 丁少华，刘畅，王健。基于 FPGA 的实时视频预处理流水线设计与实现[J]. 电子技术应用，2022, 48 (3): 56-60.
[13] 李滨，赵晓宇。基于 Zynq-7000 的异构嵌入式视觉系统设计 [J]. 微电子学与计算机，2023, 40 (2): 89-95.
[14] Hartl A, Kuhn A, Dietrich F. Real-Time Image Preprocessing on FPGAs for Autonomous Navigation[C]. IEEE International Conference on Intelligent Transportation Systems, 2022: 1023-1028.
[15] 王田苗，陶永，陈旸。我国移动机器人技术研究现状与产业化发展趋势[J]. 机器人，2021, 43 (5): 513-524.
[16] Roumeliotis S I, Bekey G A. Extended Kalman Filtering for Integrated Inertial and Visual Odometry[J]. IEEE Transactions on Robotics and Automation, 2002, 18(2): 185-197.
[17] 张毅，罗元，郑太雄。移动机器人技术基础与设计 [M]. 重庆：重庆大学出版社，2019.
[18] OpenCV Team. OpenCV: Open Source Computer Vision Library[CP/OL]. https://opencv.org, 2024.
[19] Robotics Development Team. ROS 2: Robot Operating Version 2 Documentation[CP/OL]. https://docs.ros.org, 2024.